

# Identificadores de Proceso

Malak Sanchez

Monitorias Sistemas Operativos

1 de septiembre de 2026

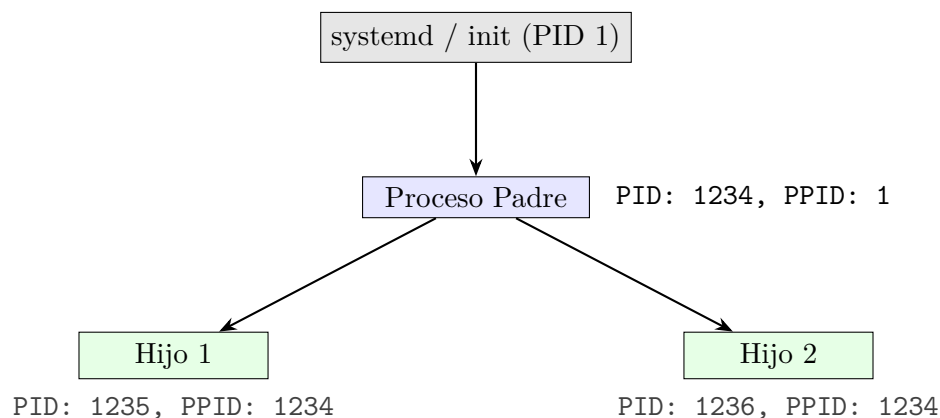
## 1. Motivación

En un sistema con cientos de procesos corriendo al mismo tiempo, el Sistema Operativo necesita una forma única de etiquetar y rastrear cada uno de ellos. Al igual que nosotros tenemos una identificación personal, cada proceso tiene un número único que permite enviarle señales, esperar por él o conocer quién lo creó.

## 2. Idea Fundamental

Todo proceso en POSIX tiene dos identificadores clave: su propio ID (**PID**) y el ID de su creador o padre (**PPID**). Estas “etiquetas” son números enteros de tipo `pid_t`.

## 3. Jerarquía de Procesos



## 4. Sintaxis Básica

```
#include <unistd.h>
#include <sys/types.h>

pid_t my_pid = getpid(); // Obtiene el PID del proceso actual
pid_t my_ppid = getppid(); // Obtiene el PID del proceso padre
```

## 5. Ejemplo Mínimo Funcional

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main() {
5     printf("Self PID: %d\n", getpid());
6     printf("Parent PID: %d\n", getppid());
7
8     if(fork() == 0){
9         printf("[CHILD] PID: %d, PPID: %d\n", getpid(), getppid());
10    }else{
11        printf("[PARENT] PID: %d, PPID: %d\n", getpid(), getppid())
12        ;
13    }
14    return 0;
15 }
```

## 6. Explicación Paso a Paso

1. El programa inicia e imprime sus propios datos.
2. Se ejecuta `fork()` creando un hijo.
3. El hijo imprime su PID (nuevo) y su PPID (que será el PID del padre).
4. El padre imprime su PID y su PPID (posiblemente la *shell* desde donde se ejecutó).

## 7. Qué ocurre en memoria

El Sistema Operativo mantiene una **Tabla de Procesos**. Cada entrada en esta tabla contiene el PID, PPID, estado de ejecución y otros metadatos (el PCB - Process Control Block). Las funciones `getpid` y `getppid` simplemente consultan esta tabla para el proceso que las llama.

## 8. Patrones comunes de uso

Rastreo de jerarquía para depuración:

```
1 void log_context(char *msg) {
2     printf("[%s] PID: %d (PPID: %d)\n", msg, getpid(), getppid())
3     ;
4 }
```

## 9. Errores Comunes

- **PPID = 1**: Si el padre termina antes que el hijo, el PPID del hijo cambiará automáticamente a 1 (adoptado por *init*).

- **Confusión con retornos:** No confundir el valor de retorno de `fork()` con el resultado de `getpid()`.

## 10. Buenas Prácticas

- No hardcodear PIDs (ej: `kill(1234, ...)`); siempre obtenerlos dinámicamente.
- Usar PIDs en los nombres de archivos temporales o *logs* para evitar colisiones entre procesos.

## 11. Ejercicios

1. Escriba un programa que cree dos hijos; uno de ellos debe crear un nieto. Cada proceso debe imprimir su PID y su PPID junto con su rol.
2. Consulte desde la terminal el PID de su intérprete con `echo $$` y contraste ese valor con el PPID que muestra un programa recién iniciado.